PickApp: A Lost and Found Mobile Application

A Case Study Project

By:

Miko D. Domingo

Hazel Kim A. Maglanoc

Josh Emil D. Palad

Areown Dapne E. Venus

Submitted To:

Mr. Angelito I. Cunanan Jr.

November 2026

Problem and Proposed Solution

Problem

Losing or finding personal belongings within a school campus is a common situation that can cause inconvenience and difficulty for students, faculty, and staff. Items such as identification cards, wallets, mobile phones, keys, notebooks, and other personal belongings may be misplaced or accidentally left in classrooms, hallways, laboratories, cafeterias, and other campus areas. When these situations occur, individuals often rely on informal methods to report or search for their missing belongings, such as asking people nearby, informing school personnel, sending messages through group chats, or checking with the campus guard house. Although these methods provide a way to communicate lost-and-found information, they do not always ensure that reports reach the people who need them.

One of the main problems with these existing practices is the lack of a centralized and organized platform for managing lost-and-found reports. Information shared through group chats can quickly become buried among other conversations, while verbal reports may not reach everyone on campus. Furthermore, reports are sometimes provided without photographs or specific location details, making it difficult for others to identify the item or determine where it was last seen. The process of connecting the person who lost an item with the person who found it also depends largely on personal communication and individual initiative. As a result, some lost items may remain unclaimed, while individuals spend additional time asking around or searching different locations. These circumstances highlight the need for a more accessible, organized, and efficient way of reporting, locating, and managing lost-and-found items within the campus.

Intended Users

PickApp is intended for students, faculty members, and school personnel who may encounter situations involving lost or found personal belongings within the campus. Students and faculty members can use the application to report items they have lost or found, browse existing reports, and communicate with individuals who may have relevant information about a particular item. School personnel, including campus guards and office staff members who temporarily keep found belongings, can also benefit from having a shared platform where information about reported items can be accessed. By providing a common space for submitting and viewing reports, the application aims to make lost-and-found information more accessible to members of the school community, regardless of whether they personally know the individual who lost or found an item.

Proposed Solution

To address these problems, the group developed PickApp, a mobile application designed to provide a centralized platform for reporting, browsing, and managing lost-and-found items within the campus. The application allows users to submit reports by providing the item's title, category, description, contact information, and location. Users can also attach photographs to help others recognize the reported belongings and use the device's camera and geolocation capabilities to provide visual and location-based information. The submitted reports are stored in a shared database through a PHP REST API, allowing users on different devices to access the same collection of lost-and-found information.

In addition to submitting reports, users can browse reported items, search using keywords, and apply filters based on item type, category, and date. The application also provides separate sections for active and resolved reports, helping users distinguish between items that are still being searched for and those that have already been returned or claimed. To support communication between individuals, PickApp includes the "I Found This" feature, which allows someone who has found an item from an existing Lost report to provide their contact information to the owner. Similarly, the "This Is Mine" feature allows individuals to submit a claim for an item listed under Found reports by providing contact information and a detail that helps establish ownership. Notifications and vibration alerts help inform the concerned users when someone reports finding their item or submits a claim. Report owners can also edit, delete, resolve, or reopen their own reports.

Through these functionalities, PickApp aims to reduce dependence on scattered communication methods and provide a more systematic approach to handling lost-and-found information. By combining item reporting, photo attachments, location identification, search and filtering, claims, and notifications in one mobile application, the system makes it easier for members of the campus community to share information and connect individuals who have lost or found belongings. Ultimately, the application seeks to improve the accessibility, organization, and convenience of the campus lost-and-found process.

Application Features

The app has three tabs, Browse, Resolved and Report, plus a details screen that opens from any report.

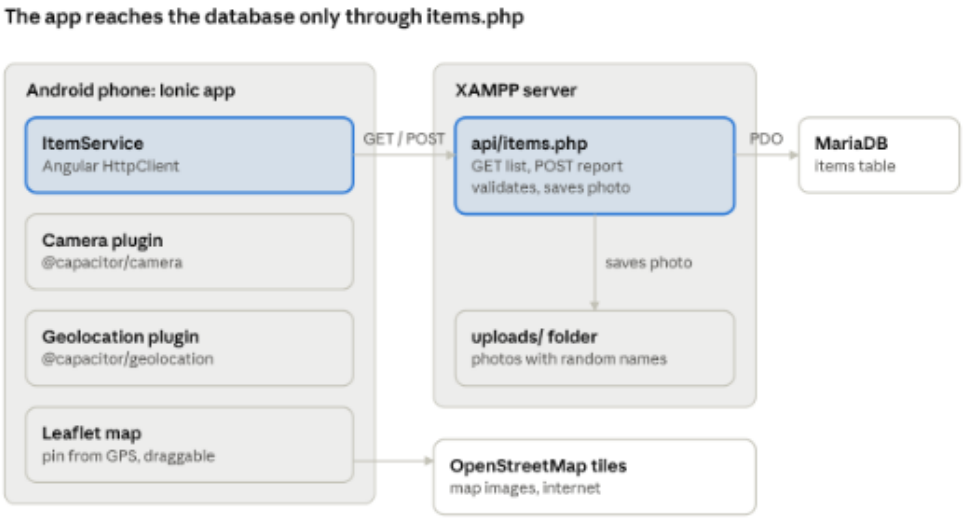
Feature	Where	How the user interacts with it
Report list	Browse tab	Opens on launch and shows every report, newest first, as a compact card: photo thumbnail (or the category's icon when there is no photo), Lost/Found label, title, category and how long ago it was reported (Today, Yesterday, 3 days ago)
Lost / Found filter	Browse tab	A segment at the top (All, Lost, Found); the list reloads with only that type
Keyword search	Browse tab	Type in the search bar under the title; the list keeps only reports with those words in the title or description
Category and date filters	Browse tab	Tap Filters (top right) and choose a category, a date range (today, past 7 days, past 30 days). Each active filter shows as a chip that removes it when tapped
Pull to refresh	Browse tab	Pull the list down to fetch the latest reports
Resolved list	Resolved tab	Shows only the reports marked resolved, with the same search, Lost / Found segment and filters; Browse shows only open reports
Item details	Opens from a card	Tap a card: full photo, description, contact (a number you can tap to call, or a Facebook name), date reported, coordinates, and a map with a red (lost) or green (found) pin

Mark as resolved	Item details	On the phone that submitted the report: Mark as resolved (after a confirmation) or Reopen report. A resolved report moves to the Resolved tab, with a Resolved badge and the date it was resolved
Edit report	Item details	On the phone that submitted the report: tap Edit. The Report form opens filled in; change any field, the contact, the photo (replace or remove) or the map pin, then tap Save changes
Delete report	Item details	On the phone that submitted the report: tap Delete and confirm. The report and its photo are removed and the list reloads
I found this	Item details	On someone else's open Lost report: tap I found this and enter a contact number or Facebook name. The report then shows "Someone says they found this" with that contact. The owner's phone vibrates on and off for up to a minute (until the alert is answered) and shows "Someone found your item!", when the app opens, comes back to the front, or within 30 seconds while it is open
This is mine (claims)	Item details	On someone else's open Found report: tap This is mine and enter a contact number or Facebook name and a detail only the owner would know (at least 5 characters). The report then shows "You claimed this". The person who found it gets "Someone says this is theirs!" with the detail, the same vibration and notification, and a Claims list on the report with every claimer's detail and contact
Notifications	Phone's notification bar; bell on Browse	A found notice also appears in the phone's notification bar (tap it to open the report). The bell at the top of Browse shows a red count of unread notices and lists every past notice; tap one to open its report

Report form	Report tab	Choose "I lost something" or "I found something", enter title, category and description, then under "How can people reach you?" pick Contact number or Facebook name and fill it in. A number must be a valid mobile number (e.g. 0917 123 4567)
Photo	Report tab	Tap Add photo, then Take photo or Choose from gallery; the photo is previewed and can be retaken or removed
Location	Report tab	Tap Get Current Location; the map shows the pin and the GPS accuracy in metres. Drag the pin or tap the map to correct it, or use Pick on map if GPS is unavailable
Submit	Report tab	Tap Submit report: a loading indicator shows, then a success message, and the app returns to Browse with the new report on top
Feedback and errors	Both tabs	Messages for missing fields, denied permissions, no GPS fix, and no connection to the server, with a Try again button on Browse. Each message also comes with a matching vibration (success, warning or error)

Technical Implementation

The app uses three Capacitor capabilities, Camera, Geolocation with Leaflet, and Haptics, plus Local Notifications for the found-item and claim alerts, and saves its data through a PHP REST API backed by MariaDB.



The app never touches the database directly. Photos and coordinates gathered by the plugins travel in one POST from ItemService to items.php.

Stack. Ionic 9 with Angular 22 (standalone components and signals), Capacitor 8, Leaflet 1.9 with OpenStreetMap tiles, PHP 8.2 with PDO, and MariaDB 10.4 on XAMPP. The backend follows the structure of the class reference backend (http-backend-main): a config.php with the PDO connection and one endpoint script that returns JSON.

Capacitor capabilities.

- **Camera** (@capacitor/camera): Camera 8 deprecates getPhoto with CameraSource.Prompt, so the app shows its own action sheet and calls takePhoto or chooseFromGallery. The plugin returns a webPath; the app fetches it into a Blob so it can be attached to FormData.
- **Geolocation** (@capacitor/geolocation): the app requests permission, then calls getCurrentPosition with high accuracy and a 15-second timeout. The latitude, longitude and accuracy are stored in Angular signals.
- **Leaflet**: a reusable map component draws OpenStreetMap tiles and a custom pin. On the Report tab the pin is draggable and the map can be tapped; on the details screen it is read-only.

- **Haptics** (@capacitor/haptics): a small HapticsService calls Haptics.notification() with Success, Warning or Error, as in the class Haptics module. The Report page and the details screen call it inside showToast(), so every message has a matching vibration. A try/catch keeps the app working on devices that cannot vibrate, and the plugin adds the VIBRATE permission to the APK by itself. When someone found an owner's item, a ring() method repeats Haptics.vibrate (1 s on, 0.5 s off) for up to a minute and returns a stop function that the alert calls as soon as it is answered.
- **Local Notifications** (@capacitor/local-notifications): when a found notice arrives, the app schedules a notification with the report's title and the finder's contact, using a white bell icon (ic_stat_pickapp). Tapping it opens the report. The app asks for the notification permission on first start (Android 13 and newer). Each notice is also saved on the phone for the bell list. The notification is posted with isExactNotification: false. It is shown at once, so it needs no exact alarm; without this setting the plugin opened Android's Alarms & reminders settings screen on Android 12 and newer.

API operations (api/items.php).

Method	Request	Result
GET	/api/items.php	All reports, newest first, as JSON
GET	/api/items.php?type=lost (or found)	Only that type
GET	/api/items.php?q=umbrella&category=Bags &status=open&from=2026-10-25T00:00:00 +08:00	Reports matching every filter given (keyword, category, status, date range); any can be left out
POST	multipart form: title, description, type, category, latitude, longitude, contact_info, optional photo	201 with the saved report and its owner token (sent only this once); 422 with a message per field when validation fails
PUT	/api/items.php?id=5 with JSON: status (resolved or open) and owner_token	200 with the updated report; 403 if the token is wrong, 404 if there is no such report

PUT	/api/items.php?id=5 with JSON: found_contact (no token)	200 with the report and its notice; 409 if it is not an open Lost report or already has a notice; 422 without a contact
POST	/api/items.php?id=5 as a multipart form: the same fields as a new report, owner_token, an optional new photo, or remove_photo=1	200 "Report updated." with the report; the old photo file is deleted when replaced or removed; 403 if the token is wrong, 404 if there is no such report
DELETE	/api/items.php?id=5 with JSON: owner_token	200 "Report deleted." and its photo file removed; 403 if the token is wrong, 404 if there is no such report
POST	/api/claims.php?item_id=5 with JSON: contact and proof (no token)	201 "Claim sent."; 409 if it is not an open Found report or that contact already claimed it; 422 without a contact or with a proof under 5 characters; 404 if there is no such report
GET	/api/claims.php?item_id=5 with the header X-Owner-Token	200 with the claims on that report, newest first; 403 if the token is wrong, 404 if there is no such report

Every response has the shape { success, message, data, errors }. The items table stores id, title, description, type (lost or found), category, latitude, longitude, image_path, contact_info, status (open or resolved), resolved_at, owner_token_hash, found_contact, found_at and created_at.

Important implementation choices.

- **Security:** all queries use PDO prepared statements. An upload is accepted only if its real content is a JPEG, PNG or WebP image of at most 5 MB; it is saved under a random name, and the uploads folder refuses to run scripts or list files.
- **Who can change a report:** there is no login, so when a report is created the API generates a random owner token, returns it once and stores only its SHA-256 hash. The app keeps the token on the phone that submitted the report, so only that phone sees Mark as resolved, Edit, Delete and the claims, and the status, edit and delete requests are refused (403) without the right token. Editing is a POST with ?id= because PHP reads

uploaded files only from POST requests.

- **Reaching the server from the phone:** the app is served from http://localhost inside Android (androidScheme: 'http') and allows cleartext traffic, so it can call the XAMPP server on the campus Wi-Fi. With the default https scheme the calls fail with "Http failure response … 0", the error shown on the emulator in class.
- **Online server:** the APK calls the API at https://pulot.alwaysdata.net/api/items.php (alwaysdata's free PHP and MariaDB plan), so it works on any Wi-Fi or mobile data. The database password is kept only on the server, in a git-ignored config.local.php. A free InfinityFree site was tried first and dropped: it answers apps with a browser check page (aes.js) instead of JSON.
- **Contact choice:** the form saves either Phone: 0917 123 4567 or Facebook: Juan Dela Cruz in the existing contact_info field, so the database and API did not change. The app checks the number format before sending, and the details screen turns a number into a tel: link that opens the dialer.
- **Telling the owner:** a phone cannot make another phone vibrate, so a FoundNoticeService on the owner's phone asks the API when the app opens, when it comes back to the front, and every 30 seconds while it is open. A new notice on one of its own reports (it holds their owner tokens) triggers the minute-long vibration, the alert and a phone notification, and is saved in the bell list with an unread mark; each notice alerts only once. The API keeps only the first notice per report (UPDATE … WHERE found_at IS NULL).
- **Claims:** claims are kept in their own claims table (one row per claim, deleted with the report, one claim per contact) and served by claims.php. Each report in the list carries a claim_count; when it rises on one of the phone's own Found reports, the same service fetches the claims with the owner token in an X-Owner-Token header and alerts once per new claim. Without the right token the claims are refused (403), because the details are meant for the finder only. The check follows Capacitor's app state rather than the page's visibility, drops a request after 15 seconds, and also runs on pull to refresh and when the bell is opened, after the first check failed on a real phone.
- **CORS:** the API answers only the app's known origins and handles the browser's OPTIONS preflight.
- **Dates:** the API returns dates in ISO 8601 UTC, so each phone shows them in its own time zone.
- **Android permissions:** INTERNET, CAMERA, ACCESS_COARSE_LOCATION and ACCESS_FINE_LOCATION, plus gallery read access on Android 12 and older. The Haptics and Local Notifications plugins add VIBRATE and POST_NOTIFICATIONS by themselves.

Testing and Evaluation

The main user flow (browse, search and filter, report, submit, view, mark as resolved, Resolved tab, edit, delete, claim) passed in the browser, and the API passed every validation and upload test; camera, GPS, vibration and the notification bar still need testing on a real phone.

#	Feature or task tested	Expected result	Actual result	Status
1	Open the Browse tab	Reports load newest first, with a photo or a placeholder	Reports listed newest first; placeholder shown for reports without a photo	Pass
2	Filter Lost / Found	Only that type is shown	Found filter showed only the found report	Pass
3	Tap a report	Details show photo, contact, local date and the map pin	All details shown; date in local time; map and green pin drawn	Pass
4	Submit an empty form	Warning lists the missing fields; nothing is sent	"Please add: a title, a category, your contact info, the location." No request sent	Pass
5	Location unavailable	Warning, and Pick on map is offered	Warning shown; Pick on map placed the pin	Pass
6	Tap or drag on the map	Pin moves and coordinates update	Pin moved; coordinates changed	Pass
7	Submit a complete report	Loading indicator, success message, back to Browse with the new report on top	All as expected; server answered 201 Created	Pass
8	Send empty	422 with one	422 with six field messages	Pass

	fields to the API	message per field		
9	Upload a text file renamed .jpg	Rejected, nothing saved	415 "Photo must be a JPEG, PNG or WebP image."; no file saved	Pass
10	Open a script or the folder listing in uploads/	Blocked	Both returned 403 Forbidden	Pass
11	Take a photo, preview, retake	Preview appears; retake replaces it	The camera successfully captured a photo, displayed the preview, and allowed the user to retake and replace the selected photo.	pass
12	Get Current Location with GPS on	Permission prompt, then pin at the phone's position with ± m accuracy	The application successfully requested location permission and captured the device's current GPS coordinates, including location accuracy.	pass
13	Submit with a photo from the phone	Photo shows on the card and in the details	The selected photo was successfully uploaded and displayed in the report card and item details.	pass
14	Pull to refresh	List reloads	The report list successfully refreshed and retrieved the latest reports from the server.	pass
15	Phone offline or server stopped	"Cannot reach the server" with a Try again button	The application displayed a connection error message and provided a Try Again option when the server could not be reached.	pass
16	Keyword search	Only reports with the word in the title or description	"library" (only in a description) returned just that report; "100%" matched as text, not as a wildcard	Pass

17	Category and date filters	Only matching reports; active filters shown as removable chips	Keys showed only the Keys report with a "Keys ×" chip; removing the chip restored the list. A future start date returned none; an invalid date gave 400	Pass
18	Mark own report resolved	Confirmation, then Resolved banner and badge	Confirm dialog, PUT 200, success message, banner with the date, Resolved badge in the list; Reopen cleared it	Pass
19	Change someone else's report	Not possible	No button on a report from another device; PUT with a wrong token returned 403	Pass
20	Resolve a report right after submitting it	The phone keeps the new report's token	After submitting from the Report tab, the new report's id was in the phone's stored tokens	Pass
21	Vibration matches the result (browser check)	Warning, success and error vibration patterns with the matching messages	Empty submit gave the warning pattern; Mark as resolved gave the success pattern; a rejected status change (403) gave the error pattern	Pass
22	Vibration on a real phone	The phone vibrates on submit, resolve and errors	The device successfully produced the corresponding vibration patterns when submitting reports, resolving items, and encountering errors.	pass
23	Contact number	An invalid number is refused; a valid one is saved and can be tapped to call	"12345" gave "Enter a valid mobile number, e.g. 0917 123 4567."; "0917 123 4567" was saved as Phone: 0917 123 4567 and the details showed a tel:09171234567 link	Pass

24	Facebook name	The name is saved and shown with the Facebook label	"Juan Dela Cruz" was saved as Facebook: Juan Dela Cruz and the details showed "Facebook name" with the Facebook icon	Pass
25	Online API answers app requests	JSON, not a browser check page, with the app's CORS headers	pulot.alwaysdata.net returned {"success":true,"data":[]} (200) and allowed the app's origin for GET and POST	Pass
26	APK on mobile data, using the online API	Reports load and a new report with a photo is saved	The installed APK successfully loaded reports and submitted a new report with a photo using mobile data.	pass
27	I found this (finder)	An invalid number is refused; a valid one is saved and shown on the report; a second notice is refused	"12345" gave a warning and kept the form open; "0917 555 1234" was saved and the report showed "Someone says they found this" with a tel: link; a second notice got 409 "Someone has already told the owner they found this."	Pass
28	Owner is alerted (browser check)	Vibration and alert within 30 seconds, only once	A notice left from outside the app was picked up by the 30-second check: 800 ms vibration and "Someone found your item!" with the finder's Facebook name; View report opened it with "Good news"; no alert again after a reload	Pass
29	Found alert between two phones	Phone B taps I found this; phone A vibrates and shows the alert	The owner's phone successfully received the found-item alert and vibration after another phone submitted an I Found This notice.	pass

30	Resolved tab	Browse shows only open reports; Resolved shows only resolved ones	Browse listed the two open reports; the Resolved tab, titled Resolved, listed only the resolved report with its Resolved badge	Pass
31	Edit own report	The form opens filled in; the changes are saved and shown; nobody else can edit	Edit report opened with the title, category and number filled in and the map pin; a new title and the Found type were saved ("Report updated.") and the details updated. With curl: a new photo replaced the old file, remove_photo=1 removed it, and a wrong token got 403	Pass
32	Delete own report	A confirmation, then the report is gone from the list; nobody else can delete	"Delete this report?" dialog, then "Report deleted."; the list reloaded without it and the phone forgot its token. With curl: a wrong token got 403, deleting it again got 404, and the photo file was removed	Pass
33	Found alert keeps vibrating (browser check)	Vibration repeats until the alert is answered, at most one minute	The alert came 22 s after the notice was sent; a 1-second vibration repeated every 1.5 s; tapping Later stopped it at once	Pass
34	Bell list	Unread count on the bell; the list opens the report	The bell showed 1; the list showed the notice with the finder's Facebook name; tapping it opened the report and cleared the count	Pass
35	Notification in the phone's notification bar	Permission asked once; the notification shows; tapping it opens the report	The application successfully displayed a notification on the device, and tapping the notification opened the corresponding report.	pass

36	Online API allows edit and delete	DELETE allowed for the app; clear errors for a missing report or id	pulot.alwaysdata.net allowed GET, POST, PUT, DELETE and OPTIONS; editing or deleting a report that does not exist got 404, a DELETE without an id got 400	Pass
37	Claim a Found report (claimer)	A claim without a detail is refused; a valid one is saved; the same contact cannot claim twice	Send claim with no detail gave "Add a detail only the owner would know (at least 5 characters)." and kept the form open; a valid claim gave "Claim sent…" and the report showed "You claimed this…". With curl: the same contact again got 409, and a claim on a Lost or a resolved report got 409	Pass
38	Finder is alerted for every claim (browser check)	One alert per claim with its detail and contact; vibration until answered; every claim listed on the report	Two claims sent from outside the app gave two "Someone says this is theirs!" alerts, each with its detail and contact; the 1-second vibration repeated until Later; the bell listed both; the report showed Claims (2) with a call link, and the finder saw Mark as resolved, not This is mine	Pass
39	Claims stay private	Only the finder can read the claims; they are removed with the report	With curl: listing the claims without the owner token got 403; deleting the report left 0 claims	Pass
40	Online API answers claims	A claim count on every report; the app's token header allowed; clear errors	pulot.alwaysdata.net returned claim_count on each report, 403 without the token, 404 for a missing report and 400 without an id, and allowed the X-Owner-Token header	Pass

41	Claim alert on a real phone	The finder's phone alerts after a claim from another phone	First try: no alert and an empty bell, although the report showed Claims (1). After the fix, an Android 17 emulator alerted 0.8 s after the app was reopened and at once when the bell was opened, and the finder's phone alerted when PickApp was opened	Pass
42	Phone notification on Android 12 and newer	The found or claim notification appears in the notification bar; no settings screen opens	The application successfully displayed the found-item or claim notification in the phone's notification bar without redirecting the user to the Alarms & reminders settings screen.	pass

Observed issues and limitations.

- GPS can be off by tens of metres indoors. The draggable pin lets the user correct it, but only if they notice.
- If the user places the pin by hand while a slow GPS fix is still loading, the GPS fix later replaces the hand-placed pin.
- Photos are sent at full resolution (JPEG quality 70), so uploads on mobile data can be slow.
- There is no login, so anyone can post, and contact info is visible to everyone.
- Only the phone that submitted a report can resolve, edit or delete it, or read its claims; clearing the app's data or reinstalling the app loses that.
- The app needs the internet and the server; nothing is cached on the phone.
- The owner is alerted about a found item, and the finder about a claim, only while PickApp is open or the next time it is opened, and each Lost report keeps only the first notice.
- On Android 12 and newer, the first APK opened the Alarms & reminders settings screen whenever it posted a found or claim notification, because the notification plugin asks for an exact alarm by default. The notification is now posted without one (isExactNotification: false); this still needs a check on a real phone.
- The API address is fixed when the APK is built, so the APK stops working if the server's address changes.

Recommended improvements

Limitation	Improvement
Resolving, editing and deleting are tied to the phone that submitted the report	Sign-in, so ownership follows the account
Public contact info, anyone can post	Sign in with the school account; show contact info only to signed-in users
Large photo uploads	Resize the photo on the phone before sending
Fixed date ranges; plain text search	A date-range picker and MySQL full-text search
No offline use	Cache the last list on the phone and queue reports made offline
Free server plan with limited storage for photos	Move the API to the school's server or a paid plan when photos fill it
Found and claim alerts only while the app is open	Push notifications (Firebase Cloud Messaging) so the owner is told even when the app is closed

AI Usage Disclosure

The group used Claude Code (Anthropic) as a coding assistant; the group reviewed and tested its output and is responsible for the final application.

- Guided us on three parts of PickApp that went beyond what our subject covered: the phone notifications and the app-state listener, the owner token that protects resolving, editing and deleting a report, and the claims feature.
- Helped us debug the app in Android Studio.
- Assisted with the code for the rest of the app (the PHP API and SQL script, the Ionic pages, ItemService and the Leaflet map). We decided what the app should do, and reviewed, tested and changed the code ourselves. Features from our lessons, like the Camera, GPS location and vibration, we built using what we learned in class.

Tools used. Claude Code

What the group reviewed, tested or modified.

- Reviewed the generated code and examined how the different parts of the application worked together.
- Ran tests on an Android phone and recorded the actual results.
- Tested the submission of lost and found item reports, including photo uploads and location details.
- Checked the search and filtering features using different keywords, categories, dates, and statuses.
- Tested the editing, deleting, and resolving of item reports.
- Tested the *I Found This* and *This Is Mine* features and checked their corresponding notifications.
- Reviewed and modified application labels, categories, and other interface elements where necessary.
- Identified errors during testing and checked whether the necessary corrections worked.
- Compared the actual test results with the expected results and documented the findings.
- Checked the submitted source code for accidentally exposed passwords, API keys, or personal information.